Design of a Specialized Tokenizer for Mathematical Text

Tongren Xiao

tongrenxiao7@gmail.com

Department of Mathematics, Sun Yat-sen University

October 10, 2026

Can Mathematical Tokenization Improve Model Capability?

Research question

Can we train tokenizers specifically for mathematical text that improve both **intrinsic tokenization** and **downstream mathematical capability**?

Question	What would count as evidence?
RQ1: tokenizer quality	Lossless compression, robustness, speed, etc.
RQ2: model capability	Mathematical tasks such as PDE/ODE solving.
Current status	Tokenizer experiments completed; downstream evaluation to follow.

Study Overview

Component	Summary
Dataset	We constructed and classified a dataset of mathematical formulas from multiple mathematical fields.
Models	We proposed two specialized mathematical tokenizers.
Baselines	We selected five tokenizers: classical BPE and Unigram methods, and tokenizers used by mature large language models.
Results	Our two tokenizers achieve leading performance on core intrinsic metrics and in the overall comparison.

Baseline Tokenizers

Category	Tokenizer	Actual ID space	Code / tokenizer source
Classical tokenizers	Byte BPE (1)	1,024 / 2,048	huggingface/tokenizers
Classical tokenizers	SP Unigram (2)	1,024 / 2,048	google/sentencepiece
General-Purpose Tokenizers	Qwen2.5 (3)	151,665	Qwen2.5-0.5B/tokenizer.json
General-Purpose Tokenizers	Llama 3 (4)	128,256	meta-llama/llama3/tokenizer.py
General-Purpose Tokenizers	o200k (5)	200,019	openai/tiktoken

- Qwen2.5: used by the Qwen2.5 model family.
- Llama 3: used by the Meta Llama 3 model family.
- o200k_base: used by GPT-4o and GPT-4.1.

SP Unigram: Training and Inference

Training step	Procedure
Preparation	Use raw strings without language-specific word segmentation; represent spaces as a symbol, usually _.
1. Initialize	Create a large seed vocabulary of characters, frequent substrings and symbols; assign initial piece probabilities.
2. EM iterations	E: sum over possible segmentations to obtain expected piece counts. M: re-estimate probabilities from those counts.
3. Prune	Remove pieces with low contribution to corpus likelihood; repeat EM and pruning to reach 1,024 or 2,048 IDs.
4. Final model	A piece vocabulary and probabilities; an optional byte fallback table.

Inference

Build a segmentation lattice; for abc: $a+b+c$, $ab+c$, $a+bc$, or abc.

$$P(s) = \prod_i p(s_i), \qquad s^* = \arg\max_{s \in \mathcal{S}(x)} P(s).$$

Viterbi dynamic programming returns the most probable token sequence. (6) (2)

SP Unigram: A Two-Round EM Example

Setup: one training string ab; pieces a, b, ab; initially $p(a) = p(b) = p(ab) = 1/3$.

E step: segmentation likelihoods and posteriors

Round	(a, b) likelihood	ab likelihood	$P(x)$	Posterior: (a, b) / ab
1	$1/3 \times 1/3 = 1/9$	$1/3 = 3/9$	$4/9$	$1/4 = 0.25$ / $3/4 = 0.75$
2	$0.2 \times 0.2 = 0.04$	0.6	0.64	0.0625 / 0.9375

M step: expected counts and normalized probabilities

Round	$\mathbb{E}[n_a], \mathbb{E}[n_b], \mathbb{E}[n_{ab}]$	Total	$p(a), p(b), p(ab)$
1	$0.25,\ 0.25,\ 0.75$	1.25	$0.2,\ 0.2,\ 0.6$
2	$0.0625,\ 0.0625,\ 0.9375$	1.0625	$1/17,\ 1/17,\ 15/17$ $\approx 0.0588,\ 0.0588,\ 0.8824$

$$P(x) = \sum_{s \in \mathcal{S}(x)} \prod_i p(s_i), \qquad p(v) = \frac{\mathbb{E}[n_v]}{\sum_u \mathbb{E}[n_u]}.$$

For this one-string example, $p(ab) \to 1$ and $p(a), p(b) \to 0$: EM favors the whole piece ab. Low-contribution pieces become pruning candidates; required characters remain protected. (6)

Our Two Routes: Contribution and Prior Art

Dimension	Math-atom BPE	Boundary AST
Vocabulary	Fit-derived complete-command/Unicode atoms; original BPE.	Exactly the same byte vocabulary and ID budget.
Segmentation	Original learned BPE merge order.	Minimum additive token/tree/boundary cost on a legal source graph.
Mathematical prior	Lexical units observed in training.	Lexical units plus a restricted source parser.
Deployment	Native reversible codec + original Rust BPE.	Fused native trie traversal and dynamic programming.
Novelty boundary	A reproducible domain-specific training recipe.	A tested structural trade-off; shortest paths and AST ideas are not new.

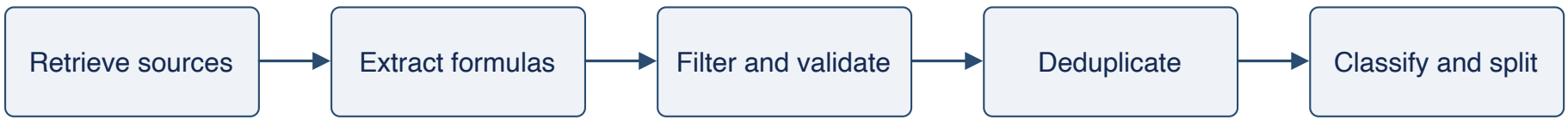
Central comparison

Holding the dictionary fixed isolates the effect of the structural segmentation rule. Comparing against generic tokenizers instead measures the complete specialized system. (7) (8) (9)

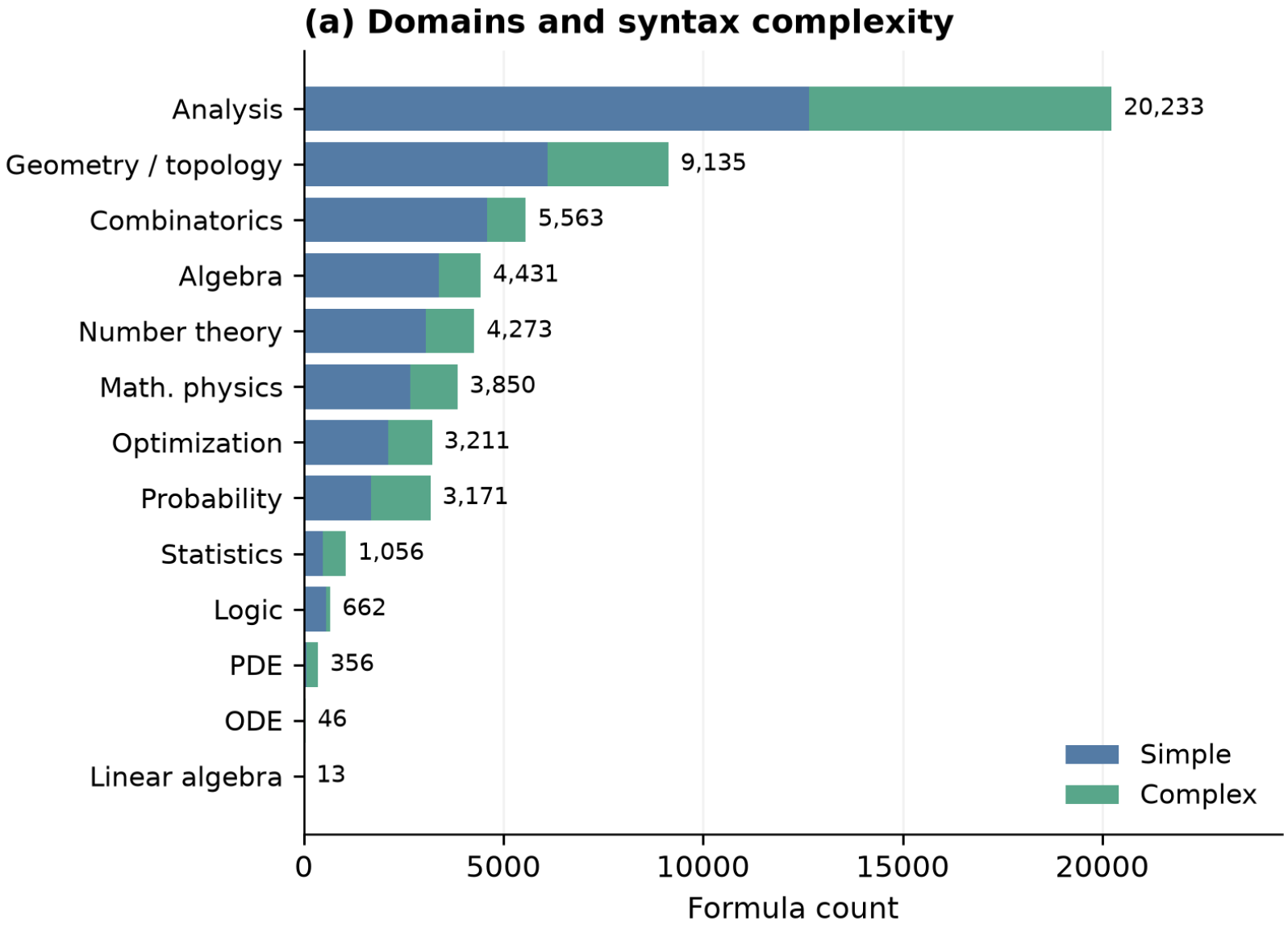
Dataset

Source	Accepted formulas	Source link
arXiv (pre-2020 papers)	51,229	arxiv.org
OpenAI math repository	2,810	github.com/openai/math
FunGrim	1,889	github.com/fredrik-johansson/fungrim
SymPy	72	github.com/sympy/sympy

Construction process



Dataset


(a) Domains and syntax complexity
Analysis
Geometry / topology
Combinatorics
Algebra
Number theory
Math. physics
Optimization
Probability
Statistics
Logic
PDE
ODE
Linear algebra
20,233
9,135
5,563
4,431
4,273
3,850
3,211
3,171
1,056
662
356
46
13
Simple
Complex
0
5000
10000
15000
20000
Formula count

Dataset

Formula types

Formula type	Count	Formula type	Count
analysis	20,233	geometry / topology	9,135
combinatorics	5,563	algebra	4,431
number / theory	4,273	mathematical / physics	3,850
optimization	3,211	probability	3,171
statistics	1,056	logic	662
pde	356	ode	46
linear / algebra	13		

Provenance labels

Provenance label	Formulas
Historical human	51,229
Possible mixed	2,810
Unknown	1,961
Fully AI generated	0

Complexity classes

Simple	Complex
37,400	18,600

Complexity score

$$s = .40\bar{L} + .20\bar{C} + .15\bar{D} + .15\bar{S} + .10\bar{O}, \quad \bar{z} = \min(z/z_0, 2).$$

L: lexemes; C: command types; D: nesting; S: symbol types; O: operators.
Normalizers $(L_0, C_0, D_0, S_0, O_0) = (60, 6, 3, 18, 12)$; complex iff $s \geq .65$.

Example: $\frac{1}{s^a}$ has $(L, C, D, S, O) = (9, 1, 1, 5, 2)$, hence $s \approx 0.202 < 0.65$: simple.

Train: 36,615 formulas; validation: 7,693; test: 11,692.

Route 1: Learning a Mathematical Lexical Alphabet

Training alphabet

$$A = \{\text{all 256 bytes}\} \cup \mathrm{Top}_{384}\{\text{observed complete commands / Unicode units}\}.$$

Atoms are derived from fit records only, ranked by frequency with deterministic tie rules. The actual primitive counts are 596 (broad) and 495 (specialized).

Stage	Implementation	Budget accounting
Reversible codec	Map observed lexical units to private-use symbols.	One primitive ID per atom; unknown strings retain byte coverage.
Vocabulary training	Original HF BPE on fit + the allowed 512-record auxiliary pool.	Total IDs capped at 1,024 or 2,048.
Inference	Native lexical codec, then original Rust BPE.	No AST path search and no policy network.

Why this route remains

It is the simple, fast mathematical reference against which extra structural machinery must justify itself. No manually enumerated command vocabulary is required. (1) (7)

Route 1: Exact Source Fidelity and Its Limits

Raw-string contract

For every evaluated x, $\mathrm{decode}(\mathrm{encode}(x)) = x$. The ordered token byte payloads also concatenate to $\mathrm{UTF8}(x)$ exactly. No whitespace normalization is permitted.

Case	Behavior	What is not guaranteed
Seen command / Unicode atom	Codec retains the complete observed unit.	A larger token can still span an argument boundary.
Unseen command	Explicit byte coverage; no unknown-token deletion.	Every novel command need not be a single token.
Spacing / multiline source	Bytes are preserved exactly.	Layout variants need not receive identical token sequences.
Numbers / identifiers	BPE learns recurrent sequences.	Numeric magnitude is not encoded continuously.

Interpretation

High command integrity reflects observed alphabet coverage, not universal LaTeX grammar coverage or a learned theorem representation. (10) (11)

Route 2: Source Structure as a Segmentation Cost

Same vocabulary, different path

An edge (a, b) matches $x_{a:b} \in V$ at permitted lexical endpoints. The restricted parser supplies source spans and matched brace-content endpoints.

Additive objective

$$z^* = \arg \min_{z \in \mathcal{P}(x,V)} \left[|z| + \lambda \sum_{e \in z} c_T(e) + \mu \sum_{e \in z} c_B(e) \right].$$

c_T: token partially crosses a native source span.
c_B: number of brace-content endpoints strictly inside the token.
Development selection: $\lambda = .25$, $\mu = .30$.

A source-tree method

The parser is restricted and falls back to source syntax. It is not a full TeX interpreter or a semantic operator tree for arbitrary mathematics. (8) (9)

Route 2: Exact Dynamic Programming on a Fixed Graph

Backward recurrence

$$D[n] = 0, \qquad D[a] = \min_{(a,b)\in E} \{1 + \lambda c_T(a,b) + \mu c_B(a,b) + D[b]\}.$$

A byte trie enumerates matching dictionary edges. A stored best endpoint reconstructs the token sequence. This is exact for the specified legal graph and additive objective.

Design	Mechanism	Reason
Lexical mask	Suppress cuts inside covered commands / codepoints.	Keep complete observed primitive units.
Tree cost	Endpoint sweeps detect partial span crossings.	Avoid testing every token against every span.
Boundary cost	Prefix sums count hidden brace-content endpoints.	Recover argument boundaries without hard blocking all compression.

Scope of optimality

The result is not globally optimal vocabulary learning, and a low source-tree cost does not imply low task loss. The shortest-path algorithm itself is established. (8) (12)

Route 2: Development Selection and Runtime Cost

Setting	Values / behavior
Prespecified grid	$\lambda \in \{0, .25, .75\}$; $\mu \in \{0, .05, .15, .30\}$: 12 candidates.
Selection criterion	Common old-dev utility plus .04 times missing parameter-boundary recall.
Feasibility guard	Compression regression at most 1%; boundary recall loss at most 1 percentage point vs. math-atom BPE.
Selected setting	All four budget/scenario conditions selected .25 / .30 before v8 scoring.
Public encoder	C frontend + trie + DP; all source parsing is included in throughput.

Engineering and trade-offs

Typical lattice work scales with input bytes, maximum piece length and source spans; the dense byte trie uses more memory than a compact sparse trie.
Compared with the older length-first path, the boundary term costs some speed but restores much of the parameter-boundary recall.

Deployment contract

A frozen dictionary and weights define deterministic IDs. Unknown lexical units use byte coverage; supported source parsing has an explicit one-million-byte input limit.

Common Inputs, Training Information and Timing

Axis	Controlled implementation
Vocabulary budgets	1,024 / 2,048 for trained methods; all primitive and reserved IDs count.
Allowed data	Identical scenario fit/auxiliary pools; generic trainers receive their union.
Test input	Original accepted raw strings; no per-model filtering, truncation or normalization.
Probability calibration	Same fit IDs for each scenario; mature models recalibrated for broad and specialized separately.
Timing	Seven shuffled/interleaved full public encode/decode runs; median throughput; caches cleared.
Statistical unit	Paper groups, not individual formulas copied across seeds.

Remaining asymmetry

Historical tuning counts and training wall time differ. Mature vocabulary budgets differ. These differences are disclosed rather than described as fully equal compute.

Efficiency, Fidelity and Information Measures

Metric	Definition / interpretation
Paper-macro tokens/byte ↓	$\frac{1}{P}\sum_p T_p/B_p$; prevents long papers from dominating the primary compression average.
Length distribution	Tokens/formula and 50th/95th/99th percentiles; inspect long-tail inflation.
Exact fidelity	Both decoded strings and concatenated raw token payloads equal the source.
Encode / decode speed	Raw UTF-8 MiB/s through the complete public interface.
Fit unigram BPB ↓	$-B^{-1}\sum_i \log_2[(c_i+\alpha)/(N+\alpha\lvert V\rvert)]$; fit-only count model, not LM perplexity.
Information / utilization	Renyi efficiency at $\alpha = 1, 2, 2.5, 3$; used IDs / actual vocabulary.

Sensitivity

Report smoothing at .01 / .1 / .5 / 1. Information efficiency and token count measure different properties; neither alone proves downstream quality. (12) (13)

Structure, Integrity and Robustness

Metric	Definition	Evidence boundary
Command / number integrity	Fraction of reference lexemes with no internal token cut.	Does not require isolation as a standalone token.
Lexical boundary P/R/F1	Token endpoints against reference lexical endpoints.	A byte tokenizer can obtain high recall.
KaTeX crossing rate	Tokens partially overlapping a reference syntax span / all tokens.	A whole-formula token can obtain low crossing.
Parameter-boundary recall	pylatexenc group-content endpoints retained as token boundaries.	Independent implementation; syntax proxy, not semantic gold.
Robustness	Layout token variation; local symbol edit effects outside a 16-byte neighborhood.	The local edit is not mathematical equivalence.

Coverage is part of the result

Unicode integrity, parser coverage and probe counts accompany the scores. No formula disappears from the common test merely because a diagnostic parser cannot measure it. (13) (11)

Three New Review Rounds of the Retained Release

Round	Checks actually executed	Scope
1. Data / provenance	Reclassify every formula; verify file IDs, hash, license/origin rules and source-disjoint pools.	56,000 catalog records.
2. Public-encoder replay	Reload every current model; reconstruct original payload; replay length, reference crossings and parameter boundaries.	234,556 formula-result rows.
3. Independent aggregation	Recompute paper averages, integrity and fit-calibrated probability bits; check common dictionary and seven timing repeats.	19 configurations, 109 metric records.

Status

All three new reviews passed. The earlier v8 freeze/evaluation additionally passed eight protocol/result checks and 201 implementation tests.

Reorganization integrity

Selected checkpoints are byte-identical copies of the frozen v8 artifacts. Current results are the seven-method subset, with no new training or test-based parameter changes.

New-Paper Results: 1,024-ID Trained Vocabulary

Common test: 4,294 formulas from 27 new source papers.

System	T/B ↓	Cross% ↓	Scope% ↑	Cmd% ↑	Num% ↑	MiB/s ↑
Boundary AST (ours)	0.3900	18.43	64.36	99.76	97.41	29.68
Math-atom BPE (ours)	0.3943	19.26	62.83	99.76	97.41	10.20
Byte BPE	0.4121	25.51	67.62	62.28	97.43	8.59
SP Unigram	0.4912	22.34	68.64	2.49	98.08	23.42
Qwen2.5	0.5318	21.62	84.76	2.16	97.41	3.14
Llama 3	0.5279	21.74	84.76	2.16	99.28	8.94
o200k	0.5246	22.24	84.91	2.19	99.28	14.32

Reading the table

T/B is paper-macro tokens per UTF-8 byte. Crossing and scope recall use different external syntax references; improving one does not imply improving the other.

Comparison scope

The two proposed systems share a dictionary. Byte BPE and Unigram share the ID budget and allowed data. Qwen2.5, Llama 3 and o200k retain their full vocabularies.

New-Paper Results: 2,048-ID Trained Vocabulary

Common test: 4,294 formulas from 27 new source papers.

System	T/B ↓	Cross% ↓	Scope% ↑	Cmd% ↑	Num% ↑	MiB/s ↑
Boundary AST (ours)	0.3296	24.17	53.87	99.76	97.89	28.65
Math-atom BPE (ours)	0.3372	24.87	51.18	99.76	97.57	9.56
Byte BPE	0.3504	31.46	59.79	74.75	97.61	8.81
SP Unigram	0.4625	24.03	67.07	4.16	98.71	23.54
Qwen2.5	0.5318	21.62	84.76	2.16	97.41	3.14
Llama 3	0.5279	21.74	84.76	2.16	99.28	8.94
o200k	0.5246	22.24	84.91	2.19	99.28	14.32

Reading the table

T/B is paper-macro tokens per UTF-8 byte. Crossing and scope recall use different external syntax references; improving one does not imply improving the other.

Comparison scope

The two proposed systems share a dictionary. Byte BPE and Unigram share the ID budget and allowed data. Qwen2.5, Llama 3 and o200k retain their full vocabularies.

Strict Source-Domain Generalization

Common holdout: 1,428 formulas. Specialized training excludes target and adjacent source categories.

System	T/B: 1,024	T/B: 2,048	Scope%: 1,024	Scope%: 2,048
Boundary AST (ours)	0.4372	0.4010	61.27	53.43
Math-atom BPE (ours)	0.4470	0.4118	60.70	53.72
Byte BPE	0.4726	0.4188	66.29	61.13
SP Unigram	0.5325	0.5011	70.89	70.10
Qwen2.5	0.5415	0.5415	86.87	86.87
Llama 3	0.5353	0.5353	86.87	86.87
o200k	0.5335	0.5335	86.63	86.63

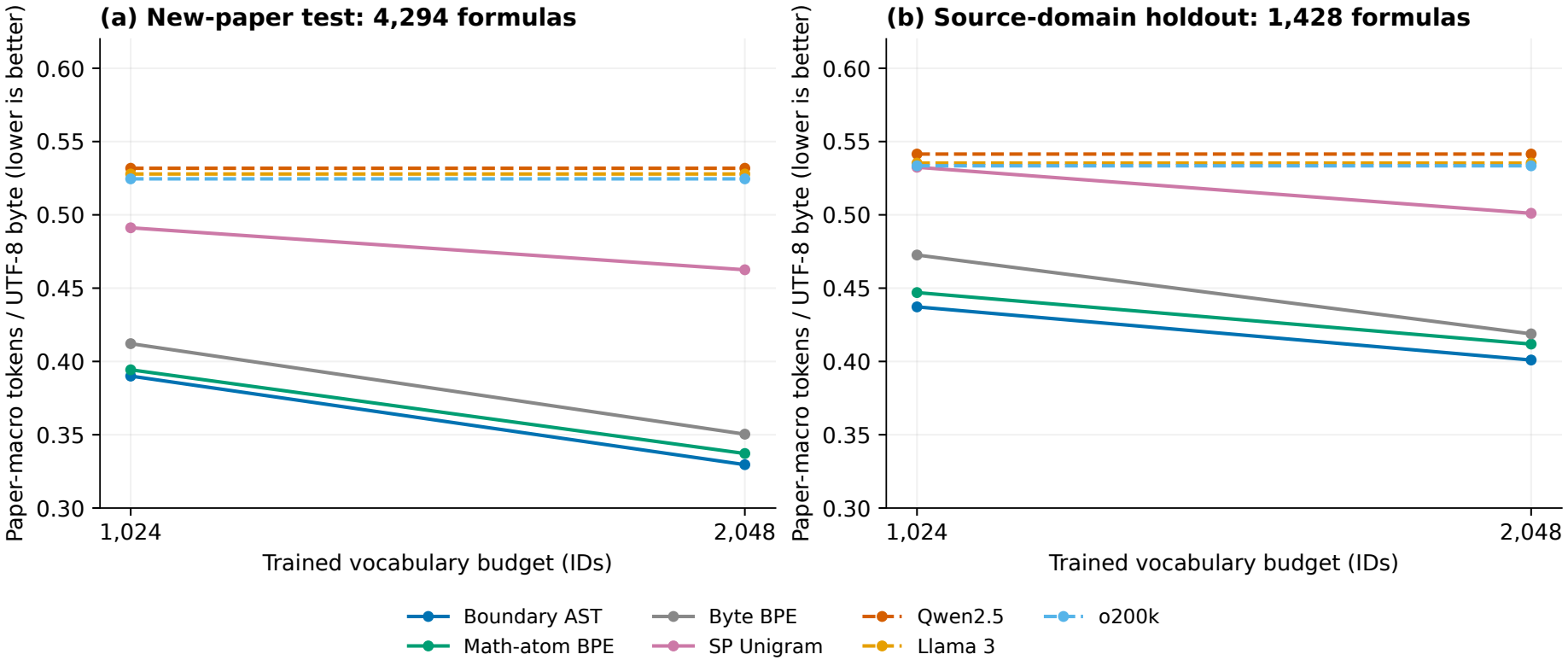
What generalizes

Boundary AST compresses more than the same-dictionary BPE in both budgets. Scope recall changes by +0.57 pp / -0.28 pp; not every structural diagnostic improves.

What remains uncertain

Source categories are disjoint by construction, but mathematical concepts can overlap. Few source papers and uneven domains produce wide uncertainty intervals.

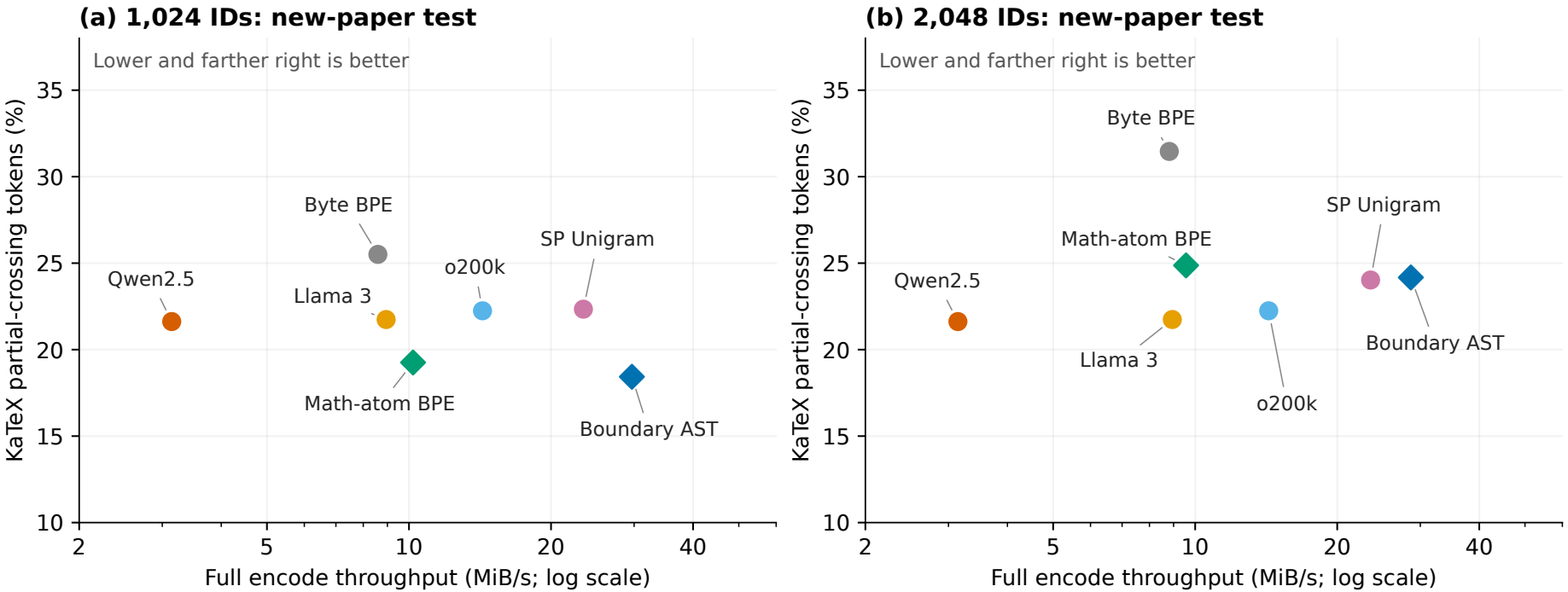
Compression Across Vocabulary Budgets



Interpretation

Solid lines: trained 1,024/2,048-ID systems. Dashed lines: fixed full-vocabulary general tokenizers. Lower is shorter; this is not a downstream-accuracy curve.

Runtime and Source-Tree Crossing



Interpretation

Boundary AST is about 2.65–3.00 times faster than math-atom BPE across the four principal conditions. All parser/codec work is included. The result is from one Mac and a single-input public interface.

Paired Paper-Cluster Statistical Evidence

Difference: Boundary AST minus math-atom BPE. Negative compression differences favor AST.

Condition	T/B difference	Family 95% CI	Scope difference CI (pp)
1,024 / broad	-0.00428	[-0.00601, -0.00270]	[-2.36, +6.06]
1,024 / domain	-0.00976	[-0.02916, -0.00197]	[-4.44, +9.88]
2,048 / broad	-0.00760	[-0.01106, -0.00535]	[+0.49, +6.59]
2,048 / domain	-0.01088	[-0.03025, -0.00344]	[-5.78, +12.17]

Design

200,000 paper-cluster resamples; Bonferroni intervals retain the original family of 144 comparisons. We did not shrink the family after choosing seven systems.

- All four adjusted compression intervals exclude zero in favor of Boundary AST.
- Scope and crossing intervals do not all exclude zero; point estimates are not universal structural gains.
- Intervals condition on the realized vocabulary and observed training seeds. They do not cover all possible retraining outcomes.

Probability Coding and Vocabulary Utilization

System	Fit BPB: 1,024	Fit BPB: 2,048	Used%: 1,024	Used%: 2,048
Boundary AST (ours)	3.131	3.008	67.3	72.0
Math-atom BPE (ours)	3.153	3.028	66.8	70.9
Byte BPE	3.413	3.209	76.3	76.3
SP Unigram	3.709	3.634	69.5	67.2
Qwen2.5	4.063	4.063	0.8	0.8
Llama 3	4.056	4.056	1.1	1.1
o200k	4.060	4.060	0.7	0.7

Interpretation

Fit BPB uses the same scenario fit records and actual vocabulary size. The much larger general vocabularies incur a different smoothing term and utilization denominator.

Why several measures are necessary

Low token count can coexist with sparse usage or unstable boundaries. These probability readouts are descriptive token statistics; no trained language-model perplexity is reported. (12) (13)

Current Seven-System Composite Ranking

System	Rank: 1,024	Rank: 2,048	Score: 1,024	Score: 2,048
Boundary AST (ours)	1	1	0.042	0.100
Math-atom BPE (ours)	2	2	0.271	0.342
Byte BPE	5	5	0.610	0.608
SP Unigram	3	3	0.433	0.450
Qwen2.5	7	7	0.860	0.817
Llama 3	6	6	0.696	0.646
o200k	4	4	0.587	0.537

Frozen metric preferences

Compression 30%; encode speed 25%; KaTeX crossing 20%; command integrity 10%; number integrity 5%; fit BPB 10%. Broad and specialized scenarios each receive 50%.

How to use the ranking

The ranking favors our complete specialized systems under this preference. It is relative to the seven retained methods, omits scope recall from its weighted score and cannot establish downstream superiority.

Limitations and Next Steps

Downstream tasks

Validate the tokenizers on PDE solving, ODE solving, and mathematical reasoning beyond equation solving.

Step	Experimental procedure
1. Architecture	Select a suitable model architecture.
2. Tokenizer integration	Connect each tokenizer to the same architecture.
3. Training	Train each model fully from scratch.
4. Evaluation	Select suitable benchmarks and evaluate downstream task performance.

Open issues

Benchmarks: identify suitable benchmarks for each mathematical task.
Compute: full training from scratch requires substantial computational resources.

References (1/2)

(1) Sennrich, Haddow and Birch (2016). Neural Machine Translation of Rare Words with Subword Units. ACL.

(2) Kudo and Richardson (2018). SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. EMNLP Demonstrations.

(3) Qwen Team (2024). Qwen2.5 Technical Report. Technical report.

(4) Dubey et al. (2024). The Llama 3 Herd of Models. Technical report.

(5) OpenAI (2026). tiktoken: BPE Tokenizer Library. Official software; accessed 2026-10-13.

(6) Kudo (2018). Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. ACL.

(7) Bai et al. (2026). Complex Mathematical Expression Recognition: Benchmark, Large-Scale Dataset and Strong Baseline. AAAI.

References (2/2)

(8) Schmidt et al. (2024). Tokenization Is More Than Compression. EMNLP.

(9) Scarlatos and Lan (2023). Tree-Based Representation and Generation of Natural and Mathematical Language. ACL.

(10) Golkar et al. (2023). xVal: A Continuous Numerical Tokenization for Scientific Language Models. Preprint.

(11) Jung et al. (2024). TeXBLEU: Automatic Metric for Evaluate LaTeX Format. Preprint.

(12) Uzan et al. (2024). Greed Is All You Need: An Evaluation of Tokenizer Inference Methods. ACL.

(13) Meister (2026). TokEval: A Tokenizer Evaluation Suite. COLM.